Lab 07 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — `localhost:5432` inside the API

Answer. Inside the `api` container, `localhost` means **the `api` container**: nobody listens on 5432 there, hence `Connection refused`. Inside the `db` container, `localhost:5432` really is PostgreSQL — but the API is not there. Fix: `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`, the container name, resolved by the user-defined network's DNS.

Why. Each container has its own network namespace, hence its own `lo`. `127.0.0.1` never crosses a namespace boundary.

Nuance. `pg_isready` inside `db` proves the database is listening, not that it is reachable from `api`. The relevant test is run **from** `api` (or from a throwaway container on the same network).

Example.

```
podman run --rm --network stack alpine nc -zv db 5432          # db (10.89.0.2:5432) open
podman run --rm --network stack alpine nc -zv localhost 5432   # Connection refused
```

Question 2 — Empty `IPAddress`

Answer. The container is on the **default rootless network**, managed by `pasta`, which copies the host's address into the container instead of assigning one: Podman therefore has no "container IP" to display. To get a `10.88.0.x` address, start the container with `--network podman` (the bridge), or better, on a network you created.

Why. An ordinary user cannot create a bridge or NAT; `pasta` does everything in user space by reproducing the host's stack. Rootless bridge networks, on the other hand, live in a private namespace of Podman's.

Nuance. `HostConfig.NetworkMode` is `pasta` in the first case, `bridge` in the second. Root Docker always shows an IP, because its default network is already a bridge.

Example.

```
podman run -d --name s alpine sleep 1d && podman inspect s --format
'{{.NetworkSettings.IPAddress}}|{{.HostConfig.NetworkMode}}'   # |pasta
podman run -d --name p --network podman alpine sleep 1d && podman inspect p --format
'{{.NetworkSettings.Networks.podman.IPAddress}}'  # 10.88.0.2
podman rm -f -t 0 s p
```

Question 3 — `ping` by IP, `ping` by name

Answer. On `podman`: `ping 10.88.0.2` succeeds (same bridge), `ping a` fails with `bad address 'a'`. On `stack`: both succeed. The difference is the `dns_enabled` field of `podman network inspect`: `false` on `podman`, `true` on a network you create — which then starts `aardvark-dns`.

Why. The bridge provides IP connectivity; name resolution is a separate service, which Podman enables only on user-defined networks (as Docker does).

Nuance. Even on `stack`, `ping` by name can fail in an image that lacks the `NET_RAW` permission; `wget`, `nc` or `getent hosts` are more reliable tests than a `ping`.

Example.

```
podman network inspect podman --format '{{.DNSEnabled}}'   # false
podman network inspect stack  --format '{{.DNSEnabled}}'   # true
podman run --rm --network stack alpine getent hosts api    # 10.89.0.3  api.dns.podman …
```

Question 4 — "`EXPOSE` is useless"

Answer. They are wrong on the substance, right on the observation. `EXPOSE` **does not publish** a port: it is image metadata saying "this service listens on 8080". It is read by `-P` (which publishes every exposed port on random host ports), displayed by `podman ps` (`8080/tcp` with no arrow) and `podman image inspect` (`ExposedPorts`), and serves as documentation for whoever deploys the image.

Why. Publishing is a **deployment** decision (which port on which machine), not an image-build decision. The image cannot make it.

Nuance. Between containers on the same network, neither `EXPOSE` nor `-p` is needed: every port of the container is reachable. `EXPOSE` remains good practice: tools (Compose, Kubernetes, scanners) rely on it.

Example.

```
podman image inspect api:1.0 --format '{{.Config.ExposedPorts}}'   # map[8080/tcp:{}]
podman run -d --name r --network stack -P api:1.0 && podman port r  # 8080/tcp -> 0.0.0.0:39843
podman rm -f -t 0 r
```

Question 5 — Port 80 refused

Answer. With classic Docker, the **root daemon** opens port 80: no restriction. Under rootless, it is your user, and the kernel reserves ports < 1024 for root. Three ways out: (1) publish on a high port, `-p 8080:80`, and accept the URL with `:8080`; (2) lower the system limit, `sudo sysctl net.ipv4.ip_unprivileged_port_start=80` (persistent in `/etc/sysctl.d/`), which opens it to every user; (3) put a reverse proxy or load balancer in front, one that runs as root or as a system service. In the workplace: (3) — an application container never listens on 80/443 directly.

Why. The restriction is a Linux kernel rule that predates containers: a privileged port used to guarantee that the service had been started by the administrator.

Nuance. The container is created but not started: `podman ps -a` shows it as `Created`. You must remove it before starting again with the same name.

Example.

```
podman ps -a --filter name=web --format '{{.Status}}'   # Created
podman rm web && podman run -d --name web -p 8080:80 nginx:alpine
```

Question 6 — `502` after recreating the API

Answer. `502 Bad Gateway`. nginx resolved `api` **at startup** to `10.89.0.3` and keeps that address. The new `api` got a different IP (`other` took the next one), the old one no longer exists: `connect() failed (113: Host is unreachable)`. The throwaway container resolves `api` at request time and finds the right address. Fix: `podman restart web` (nginx resolves again at startup), or an nginx configuration with `resolver 10.89.0.1 valid=10s;` and `set $upstream http://api:8080; proxy_pass $upstream;`, which forces a lookup on every request.

Why. By default, nginx resolves the names of static `proxy_pass` directives when it reads its configuration, once, so as not to pay a DNS query per connection.

Nuance. As long as `api` is merely **stopped**, the code is `504 Gateway Time-out` (the IP no longer answers); it is the **recreation with a different IP** that yields `502`. Spring Boot to PostgreSQL has no such flaw: every new connection from the pool resolves again.

Example.

```
podman logs web 2>&1 | grep -i error | tail -1     # upstream: "http://10.89.0.3:8080/…"
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'   # 10.89.0.12: a different address
podman restart -t 1 web && curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health   # 200
```

Question 7 — The client address is the container

Answer. Under rootless, the published port is held by `rootlessport`, a user-space relay that **accepts the connection on the host and opens a new one** to the container, from inside Podman's network namespace: nginx therefore sees the container's own address as the client. With root Docker, the port is relayed by kernel NAT rules, which preserve the real source address. Consequence: any logic based on the client IP (logging, rate limiting, allow-lists) is blind behind a rootless published port.

Why. An application-level relay (TCP proxy) terminates the connection, whereas NAT only rewrites packet headers.

Nuance. On the default network (`pasta`), the source address is better preserved. And in production, the problem is solved by the `X-Forwarded-For` header added by the load balancer or the front reverse proxy.

Example.

```
podman logs web 2>&1 | grep GET | tail -1 | cut -d' ' -f1   # 10.89.0.10
podman inspect web --format '{{.NetworkSettings.Networks.stack.IPAddress}}'   # 10.89.0.10
```

Question 8 — Two networks

Answer. Yes. Typical case: an `admin` network for monitoring or backup tools, to which you connect `api` (or `db`) without `web`. Commands: `podman network create admin`, `podman network connect --alias backend admin api`, `podman network disconnect admin api` to undo. The inspect output then shows two keys, `stack` and `admin`, each with its own `IPAddress` (`10.89.0.12` and `10.89.1.3`) and its aliases.

Why. Each network is a separate bridge; connecting a container adds an interface (`eth1`) in its namespace. Each network's DNS knows only its own members.

Nuance. A container on `admin` does **not** see `web` or `db`: the segmentation is real. It is an isolation tool more than a convenience.

Example.

```
podman network connect --alias backend admin api
podman inspect api --format '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}=
{{$v.IPAddress}} {{end}}'   # admin=10.89.1.3 stack=10.89.0.12
podman run --rm --network admin alpine wget -qO- http://backend:8080/actuator/health
```

Question 9 — `--network host`

Answer. (1) **No more** `-p`: the container listens directly on the host's ports, so two APIs with `--network host` on 8080 clash. (2) **No more internal DNS or isolation**: `api` no longer resolves `db` by name, and the container sees every interface and every port of the host. Legitimate for a network tool (`tcpdump`, scanner, monitoring agent) or for extreme network performance. Under rootless, an application that wants port 80 fails, like any process of your user: `bind() to 0.0.0.0:80 failed (13: Permission denied)`.

Why. `--network host` creates no network namespace: the process shares the host's, with the same constraints and the same conflicts.

Nuance. What the container sees under rootless `--network host` is the namespace of **your user** on the host, not a daemon's: in practice identical to a process started without a container.

Example.

```
podman run --rm --network host alpine ip -4 -o addr show | grep -v ' lo '   # your WSL
interfaces
podman run --rm --network host nginx:alpine sh -c 'nginx; sleep 1'          # bind() …
Permission denied
```

Question 10 — Network or pod

Answer. On the `stack` network, nginx writes `proxy_pass http://api:8080/` (name resolved by DNS), and `-p 18080:80` goes on `web`. In the `stackpod` pod, nginx writes `proxy_pass http://localhost:8080/` (same namespace), and `-p 18090:80` is declared **when the pod is created**; `podman ps --pod` shows a `POD ID` column and an `…-infra` container that carries the publication. This second model is the **Kubernetes** one: a Pod there is a group of containers sharing network and `localhost`, and the manifests in part 2 of this series rely on it.

Why. The pod shares a single network namespace among its members; the network links separate namespaces.

Nuance. Inside a pod, two containers cannot listen on the same port: one `8080` per pod. And the pod's `/etc/hosts` lists all its containers on `127.0.0.1`.

Example.

```
podman pod create --name stackpod -p 18090:80
podman run -d --pod stackpod --name api-pod api:1.0
podman exec api-pod cat /etc/hosts | tail -2    # 127.0.0.1 api-pod …
```

Question 11 — DBeaver on `db`

Answer. `-p 127.0.0.1:5432:5432`: the port is open only on WSL's loopback interface. The browser or DBeaver on Windows reaches it anyway because WSL 2 **forwards to `localhost` on the Windows side** every port the distribution listens on, including those bound to `127.0.0.1`. Other machines on the network see nothing. In production, you never publish the database: it is reachable only by the services on its network, and humans go through a bastion or a dedicated administration tool.

Why. Publishing a port opens a door on the machine; `127.0.0.1:` reserves it for local processes. The WSL forwarding is a Windows convenience, not a Podman rule.

Nuance. In WSL's *mirrored* networking mode, the Windows and WSL interfaces are shared, and a `-p 0.0.0.0:5432:5432` would expose the database to the local network. Keep `127.0.0.1:` as a reflex.

Example.

```
podman run -d --name db --network stack -p 127.0.0.1:5432:5432 -e POSTGRES_PASSWORD=secret
postgres:16-alpine
podman port db   # 5432/tcp -> 127.0.0.1:5432
```

Question 12 — `network rm` refused

Answer. `-f` first removes **every container and pod** connected to the network, then the network. In a clean-up, that is a bad idea because you do not read the list of what is about to disappear — and because a container can belong to that network without you having it in mind (an administration tool, a test database). Correct sequence: `podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'` to look, `podman rm -f -t 0 web api db` by name, then `podman network rm stack`.

Why. A network with members is a resource in use; Podman protects it by default, as it does for `volume rm`.

Nuance. A **stopped** container still counts as a member: the removal fails even when nothing is running.

Example.

```
podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'   # api db web
podman rm -f -t 0 web api db && podman network rm stack
```

Question 13 — Reverse proxy versus CORS

Answer. The browser applies CORS when a page calls **another origin** (scheme + host + port). If the page comes from `localhost:18080` and the API from `localhost:8080`, the ports differ: different origin, CORS headers required. With nginx serving `/` and relaying `/api/` to `api:8080`, the browser only ever talks to `localhost:18080`: same origin, no CORS. Other benefits: the API is

no longer published (smaller attack surface), a single place for TLS, compression, caching and logs, and the same `/api/` URL in development and in production. The line that does the work: `proxy_pass http://api:8080/;` inside a `location /api/` block.

Why. CORS is a **browser** policy, not a server one: a server-side proxy, which is not a browser, is not subject to it.

Nuance. The trailing `/` in `proxy_pass http://api:8080/` strips the `/api/` prefix before forwarding: `/api/actuator/health` becomes `/actuator/health`. Without that `/`, the API would receive `/api/actuator/health` and answer `404`.

Example.

```
curl -s localhost:18080/api/actuator/health     # {"status":"UP"} — through nginx, no CORS header
podman logs web 2>&1 | grep 'GET /api/' | tail -1
```